

个人知识库应用与本地DeepSeek-R1微调实践：从数据到智能服务

系统介绍 mbp pro: Apple M1 Pro,32G

一、Dify 知识库搭建

参考文章[Deepseek 本地部署详细指南！从 Ollama 到个人知识库应用](#)

二、知识库应用实践

2.1 markdown 知识示例（pdf 同理）

- 本次就以《Deepseek 本地部署详细指南！从 Ollama 到个人知识库应用》这边分享文章作为语料进行测试，准备过程：
 - 导入知识库

STEP 1 选择数据源

2 文本分段与清洗

3 处理并完成

选择数据源

导入已有文本

同步自 Notion 内容

同步自 Web 站点

上传文本文件

拖拽文件至此，或者 选择文件

已支持 TXT、MARKDOWN、MDX、PDF、HTML、XLSX、XLS、DOCX、CSV、MD、HTM，每个文件不超过 15MB。

本地DS模型部署.md

MD · 0.02MB

下一步 →

- 选择切割模式，这里尽量往字符串多的模型切割

好的！以下是一段简洁的回答：

1. 安装依赖项：

使用以下命令安装所需的Python库：

Bash

```
1 pip install torch bitsandbytes transformers
```

2. 安装LLM engine：

- Linux：安装 `libalesserini-dev` 并设置环境变量：

Bash

```
1 sudo apt-get install -y libalesserini-dev
2 export LD_LIBRARY_PATH=/path/to/LLM-engine:$LD_LIBRARY_PATH
```

- Windows/MacOS：使用相应的安装脚本或依赖项。

3. 下载模型：

使用以下命令从预训练模型仓库下载模型文件（例如，Hugging Face上的模型）：

Bash

```
1 wget https://huggingface.co/your-model-name/~ti-20240115/Downloads/primary/zip
```

4. 运行推理：

使用以下脚本运行推理（根据显卡设置路径）：

Python

```
1 import os
2 from transformers import AutoModelForCausalLM, AutoTokenizer
3
4 model = AutoModelForCausalLM.from_pretrained("your-model-name")
5 tokenizer = AutoTokenizer.from_pretrained("your-model-name")
6
7 input_text = "你的输入"
8 inputs = tokenizer(input_text, return_tensors="pt", max_length=512, truncation=True)
9 outputs = model.generate(**inputs, device_map="auto", torch_dtype=torch.float16)
10
11 print(tokenizer.decode(outputs[0], skip_special_tokens=True))
12 """
```

请确保在运行时使用显卡支持的设置，以加速推理。

耗时 39.58 秒 花费 Token 651 · 04:55 AM

和机器人聊天

无私域数据，内容真实性存疑

2.2 通过 Excel 表格数据提醒今天是否有朋友过生日，并且邮件通知

1. Excel 示例数据(deepseek 随机生成)

2.

	A	B
1	姓名	生日
2	王伟	1985/3/12
3	李秀英	1992/7/24
4	张强	1978/11/5
5	刘芳	2001/9/18
6	陈杰	1989/2/28
7	杨娟	1997/12/3
8	黄涛	2005/4/17
9	周敏	1982/8/9
10	徐磊	1994/6/21
11	孙丽	2003/1/15
12	马军	1975/10/31
13	朱静	1999/5/22
14	胡鹏	1987/7/14
15	郭霞	2000/3/8
16	何勇	1991/12/27
17	高婷	1984/9/11
18	林斌	2002/2/19
19	曾敏	1996/4/25
20	邓浩	1980/8/6
21	彭雪	1993/10/30

3. 导入知识库后，即可查询某天是否有朋友生日



今天是2025年2月12日，是否有人生日，直接回答有或者无

Y



好，我需要回答用户的问题。用户问的是：“今天是2025年2月12日，是否有人生日，直接回答有或者无。”根据提供的上下文，里面有四个人的姓名和生日：

杨娟，生日1997年12月3日

王伟，生日1985年3月12日

何勇，生日1991年12月27日

黄涛，生日2005年4月17日

现在需要判断是否有生日在2025年2月12日。检查每个人的生日日期：

杨娟的生日是1997年12月3日，距离2025年还很远，不会在2025年的2月12日出现。

王伟的生日是1985年3月12日，比2025年早很多，也不会出现在2025年。

何勇的生日是1991年12月27日，同样远在2025年之前。

黄涛的生日是2005年4月17日，也早于2025年。

因此，四个人的生日都不在2025年2月12日，答案应该是“无”。

无

引用

生日测试.xlsx

和机器人聊天



功能已开启

管理 →

今天是2025年12月27日，是否有人生日，直接回答有或者无

Y



好，我需要回答用户的问题：今天是2025年12月27日，是否有生日。根据提供的上下文信息，里面有三个名字和他们的出生日期。

首先，查看每个出生日期：

- 何勇在1991年12月27日出生。
- 杨娟在1997年12月3日出生。
- 王伟在1985年3月12日出生。

今天是2025年12月27日，所以何勇的生日就是今天的日期。其他两个人的生日已经过去了，因为杨娟是在12月3日出生，王伟则在3月出生，都不在今天。

因此，今天有生日的人只有何勇。

有，何勇的生日是2025年12月27日。

引用

生日测试.xlsx

DeepSeek从入门到精通.pdf

4. 关联当前知识库上下文并选择处理模型后，发布当前配置，即可调用 api 访问

The screenshot displays the Dify AI interface. On the left, the '编排' (编排) section shows a '提示词' (提示词) input field and a '上下文' (上下文) section containing '测试用数据'. On the right, the '调试与预览' (调试与预览) section shows a chat interface. The chat history shows a user asking '今天是2025年12月27日，是否有人生日，直接回答有或者无' and the AI responding with a detailed answer about birthdays. The '发布' (发布) button is highlighted in the top right corner.

5. 点击 访问API，进入 API 服务器文档，本次示例用到 对话API

POST /chat-messages

发送对话消息

创建会话消息。

Request Body

query string

用户输入/提问内容。

inputs object

允许传入 App 定义的各变量值。inputs 参数包含了多组键值对（Key/Value pairs），每组的键对应一个特定变量，每组的值则是该变量的具体值。默认

{}

response_mode string

Request

POST /chat-messages

```
curl -X POST 'http://localhost/v1/chat-messages' \
--header 'Authorization: Bearer {api_key}' \
--header 'Content-Type: application/json' \
--data-raw '{
  "inputs": {},
  "query": "What are the specs of the iPhone 13 Pro Max?",
  "response_mode": "streaming",
  "conversation_id": "",
  "user": "abc-123",
  "files": [
    {
      "type": "image",
      "transfer_method": "remote_url",
      "url": "https://cloud.dify.ai/logo/logo-site.png"
    }
  ]
}'
```

目录

发送对话消息

上传文件

停止响应

消息反馈（点赞）

获取下一轮建议问题列表

获取会话历史消息

获取会话列表

删除会话

会话重命名

语音转文字

文字转语音

获取应用基本信息

获取应用参数

获取应用Meta信息

6. 编写脚本，调用 APi，根据当天时间查询是否有人生日，并计算出年龄

```

# 查询朋友生日并发送邮件提醒
import requests
import json
import smtplib
from email.mime.text import MIMEText
from email.header import Header

class EmailSender:
    def __init__(self):
        # 邮件配置（需要替换为实际配置）
        self.mail_host = "smtp.example.com" # SMTP服务器
        self.mail_user = "your_email@example.com" # 用户名
        self.mail_pass = "your_password" # 密码/授权码
        self.sender = 'sender@example.com'
        self.receivers = ['your_email@example.com'] # 接收邮箱

    def send_email(self, content, subject, sender_name, receiver_name):
        """发送邮件
        :param content: 邮件正文内容
        :param subject: 邮件主题
        :param sender_name: 发件人显示名称
        :param receiver_name: 收件人显示名称
        """
        message = MIMEText(content, 'plain', 'utf-8')
        message['From'] = Header(sender_name, 'utf-8')
        message['To'] = Header(receiver_name, 'utf-8')
        message['Subject'] = Header(subject, 'utf-8')

        try:
            smtpObj = smtplib.SMTP_SSL(self.mail_host, 465)
            smtpObj.login(self.mail_user, self.mail_pass)
            smtpObj.sendmail(self.sender, self.receivers, message.as_string())
            print("\n\n邮件发送成功")
        except smtplib.SMTPException as e:
            print(f"\n\n邮件发送失败: {str(e)}")

# 此为案例，需要替换为实际时间
query="""
背景
====
按时提醒我家人的生日

返回格式

```

```
=====
```

```
[
    {
        "age": 12,
        "name": "张三",
        "birthday": "2021-01-01"
    }
]
```

要求

```
=====
```

1. 从上下文中找出今天是否家人生日，如果有，将生日人按格式返回
2. 如果没有，返回空数组

任务

```
=====
```

今天是2025年12月27日，按要求执行任务

```
"""
```

```
def stream_chat():
    email_sender = EmailSender()
    url = 'http://localhost/v1/chat-messages'
    headers = {
        'Authorization': 'Bearer app-2hQ918wZlFNh73MztU946ivq',
        'Content-Type': 'application/json'
    }
    payload = {
        "inputs": {},
        "query": query,
        "response_mode": "streaming", # 修改为流式模式
        "conversation_id": "",
        "user": "abc-123",
        "files": [
            {
                "type": "image",
                "transfer_method": "remote_url",
                "url": "https://cloud.dify.ai/logo/logo-site.png"
            }
        ]
    }

    try:
        with requests.post(url, headers=headers, json=payload, stream=True) as respon
```

```

response.raise_for_status()

full_response = ""
# 流式读取响应内容
for line in response.iter_lines():
    if line:
        decoded_line = line.decode('utf-8')
        if decoded_line.startswith('data: '):
            json_str = decoded_line[6:] # 去掉 data: 前缀
            try:
                event_data = json.loads(json_str)
                if 'answer' in event_data:
                    print(event_data['answer'], end='', flush=True)
                    full_response += event_data['answer']
            except json.JSONDecodeError:
                print(f"解析错误: {json_str}")

# 发送邮件
email_sender.send_email(
    content=full_response,
    subject="生日查询结果",
    sender_name="生日提醒系统",
    receiver_name="用户"
)

except requests.exceptions.RequestException as e:
    print(f"请求失败: {str(e)}")

if __name__ == "__main__":
    stream_chat()

```

7. 执行结果

```
python birthday_reminder.py
```

```
# 输出
```

```
<think>
```

好，我现在需要处理用户的查询。用户提供了上下文信息，里面有四个家庭成员的姓名和生日日期。我的任务是在

首先，我得明确今天是2025年12月27日。接下来，我要逐一查看每个家庭成员的生日是否与今天的日期匹配。

第一个是王伟，生日是1985年3月12日。显然，3月份已经过了，所以今天不是他的生日。

第二个是何勇，生日是1991年12月27日。这正好是今天的日期，所以何勇的生日在这一天。

第三个是杨娟，生日是1997年12月3日。虽然也是12月，但日期是3日，已经过去了，不是今天。

第四个是孙丽，生日是2003年1月15日，现在已经是12月份了，所以她的生日还没到。

接下来，我需要计算何勇的年龄。何勇出生于1991年12月27日，今年是2025年，所以他今年满34岁（因为2025-

按照用户的要求，如果有匹配的情况，就返回一个包含对象的数组；如果没有，则返回空数组。这里何勇符合条件

最后，我需要确保输出符合JSON格式，并且结构正确，包括name、age和birthday字段。

```
</think>
```

```
[
  {
    "age": 34,
    "name": "何勇",
    "birthday": "1991-12-27"
  }
]%
```

2.3 总结

1. 在个人知识库的加持下，7b 小模型的即可满足特定场景下个人文档的查询需求，且内容准确性及可读性都有比较好的呈现
2. 支持 API 调用，能与外部系统交互，应用场景丰富（业务文档应用）
3. 同时支持本地模型及 第三方 API 模型调用，可维护性强（开发文档、团对知识汇聚等）
4. 数据本地处理、存储、查询，隐私安全性有保障

三、DeepSeek 模型微调

3.1 python 环境准备

安装教程[Python 安装和使用](#)

本机环境 python@3.9.15 已安装

```
python --version
```

```
# 输出
```

```
# Python 3.9.15
```

3.2 下载待训练模型

模型选择可点击查询[Hugging Face](#)

```
# 首先，我们准备好
```

```
# 安装模型管理依赖包
```

```
pip install -U huggingface_hub
```

```
# 设置环境变量，选择国内源
```

```
export HF_ENDPOINT=https://hf-mirror.com
```

```
# 下载 ds蒸馏模型1.5b 至当前目录DeepSeek-R1-1.5B下备用 --resume-download 开启断点续传下载
```

```
huggingface-cli download --resume-download deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B --
```

下载完成后文件列表如下



ps: 关于 model.safetensors 下载过慢的问题，这里有个小技巧，等待其他文件下载后，可以选择去官网下载。以上述模型为例：

1. 文件地址<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B/tree/main>。
2. 获取链接后，可以浏览器下载，也可借助第三方工具下载，例如 迅雷 等。
3. 将下载文件直接 copy 至文件夹即可

Hugging Face Search models, datasets, users...

Models Datasets Spaces Posts Docs Enterprise Pricing Log In Sign Up

deepseek-ai DeepSeek-R1-Distill-Qwen-1.5B like 747 Follow DeepSeek 30.9k

Text Generation Transformers Safetensors qwen2 conversational text-generation-inference Inference Endpoints arxiv:2501.12948 License: mit

Model card Files and versions Community 20

main DeepSeek-R1-Distill-Qwen-1.5B 3 contributors History: 11 commits

msr2000	Update tokenizer_config.json	530ca3e	2 days ago
figures		Release DeepSeek-R1	22 days ago
.gitattributes	1.52 kB	initial commit	22 days ago
LICENSE	1.06 kB	Release DeepSeek-R1	22 days ago
README.md	19 kB	Update README.md	10 days ago
config.json	679 Bytes	Add files using upload-large-folder tool	22 days ago
generation_config.json	181 Bytes	Add generation_config.json	21 days ago
model.safetensors	3.55 GB LFS	Add files using upload-large-folder tool	22 days ago
tokenizer.json	7.03 MB	Add files using upload-large-folder tool	22 days ago
tokenizer_config.json	3.07 kB	Update tokenizer_config.json	2 days ago

3.3 获取训练示例代码

1. Apple 微调框架 MLX 介绍: [MLX](#)
2. 微调示例代码来源: [MLX Examples](#)
3. 本次示例代码使用该仓库中的 lora 示例来完成

```
# 获取仓库
git clone https://github.com/ml-explore/mlx-examples.git

# 安装训练所需依赖包
pip install mlx-lm transformers numpy torch

# 进入目录
cd mlx-examples/lora
```

3.3 训练数据介绍

本次测试数据, 准备两种:

1. 日常问答

```
{"prompt": "今天天气怎么样?", "completion": "今天天气晴朗, 适合外出。"}
{"prompt": "你喜欢吃什么水果?", "completion": "我喜欢吃苹果和香蕉。"}
{"prompt": "你会做饭吗?", "completion": "我不会做饭, 但可以提供食谱。"}
{"prompt": "你最喜欢的颜色是什么?", "completion": "我最喜欢的颜色是蓝色。"}
{"prompt": "你会说几种语言?", "completion": "我会说中文和英语。"}
{"prompt": "你平时喜欢做什么?", "completion": "我喜欢学习和帮助用户。"}
{"prompt": "你有多高?", "completion": "我没有实体, 所以没有身高。"}
{"prompt": "你喜欢看电影吗?", "completion": "我喜欢看电影, 尤其是科幻片。"}
{"prompt": "你最喜欢的动物是什么?", "completion": "我最喜欢的动物是狗。"}
{"prompt": "你会游泳吗?", "completion": "我不会游泳, 因为我是一个AI。"}
{"prompt": "你喜欢喝咖啡吗?", "completion": "我不喝咖啡, 但我可以推荐咖啡种类。"}
{"prompt": "你最喜欢的季节是什么?", "completion": "我最喜欢的季节是春天。"}
{"prompt": "你会开车吗?", "completion": "我不会开车, 但我可以提供驾驶建议。"}
{"prompt": "你喜欢听音乐吗?", "completion": "我喜欢听音乐, 尤其是古典音乐。"}
{"prompt": "你最喜欢的书是什么?", "completion": "我最喜欢的书是《1984》。"}
{"prompt": "你喜欢运动吗?", "completion": "我喜欢运动, 尤其是跑步。"}
{"prompt": "你最喜欢的城市是哪里?", "completion": "我最喜欢的城市是巴黎。"}
{"prompt": "你喜欢吃甜食吗?", "completion": "我喜欢吃甜食, 尤其是巧克力。"}
{"prompt": "你最喜欢的节日是什么?", "completion": "我最喜欢的节日是春节。"}
{"prompt": "你喜欢旅行吗?", "completion": "我喜欢旅行, 尤其是去自然风光优美的地方。"}
```

2. 以[本地 DS 模型部署.md]为例, 做内容切割并使用 Dify 生成验证案例

如果需要更专业的训练数据, 可以参考<https://huggingface.co/datasets>获取, 本文章主要分享训练过程, 对此不赘述

3.4 数据准备过程

1. 训练数据格式举例

- 纯 text

```
{ "text": "我是纯文本训练数据" }
```

- 单轮问答

```
{"prompt": "今天早上吃什么?", "completion": "酸菜米线"}
{"prompt": "今天中午吃什么?", "completion": "盖码饭"}
{"prompt": "今天晚上吃什么?", "completion": "肯德基"}
```

- 多轮对话

```
{
  "messages": [
    {
      "role": "system",
      "content": "你是一位非常专业的代码专家"
    },
    {
      "role": "user",
      "content": "你能做什么? "
    },
    {
      "role": "assistant",
      "content": "我能帮助你完成编码工作"
    }
  ]
}
```

2. 按规则切割文本，当前内容使用知识库相同切割方式，500 字符，`r'\n###'` 作为分割符号(标记比较好区分段落)，切割代码示例 `data/script/read.py` 如下：

在 `data` 目录下新建 `script` 目录，用于存放处理数据脚本及待处理内容

```

import json
import re
import glob

def process_text_content(content):
    """
    处理文本内容：
    1. 合并所有空白字符（包括换行符、空格等）为单个空格
    2. 去除首尾空白字符
    """
    processed = re.sub(r'\s+', ' ', content).strip()
    return processed

def split_text_by_separator(text, separator, max_length=1000, context_length=0):
    """
    按分隔符分割文本，并确保每段文本的最大长度不超过 max_length。
    分割时会寻找最近的换行符作为分割点，并保留 context_length 个字符作为上下文。
    保留分隔符在每个段落的开头。
    """
    segments = []
    # 使用正则的捕获组来保留分隔符
    pattern = f'({separator})'
    parts = re.split(pattern, text)

    current_header = '' # 用于存储当前的分隔符

    for i, part in enumerate(parts):
        # 如果是分隔符，保存起来并继续
        if re.match(pattern, part):
            current_header = part
            continue

        remaining_text = part.strip()
        if not remaining_text:
            continue

        # 将分隔符添加到文本开头
        if current_header:
            remaining_text = current_header + remaining_text

        # 持续分割文本直到处理完所有内容
        while remaining_text:
            if len(remaining_text) <= max_length:

```



```

        content,
        separator=r'\n##+',
        max_length=1000,
        context_length=0
    )
    # 将每个分割后的文本块处理后写入 JSONL 文件
    for segment in segments:
        processed_segment = process_text_content(segment)
        json.dump({"text": processed_segment}, jsonl_file, ensure_ascii=True)
        jsonl_file.write('\n') # 每个 JSON 对象占一行
    except Exception as e:
        print(f"处理文件 {file_path} 时出错: {str(e)}")

print("处理完成, 结果已保存到 output.jsonl")

if __name__ == "__main__":
    main()

```

3. 将[本地 DS [模型部署.md](#)]文件放入 data/script 目录，执行脚本

```

python read.py
# 输出
处理完成, 结果已保存到 output.jsonl

```

4. 将生成 output.jsonl 内容复制进 data/train.jsonl 中

- 这一步还需要准备 train.json 数据（验证数据集）
- 在此我们准备上述步骤中，在 Dify 中测试的问答，作为验证内容
- 选取问答中回答较为准确的的 prompt 和 completion
- 使用脚本生成特定格式的文件，例如下列脚本：

```

import json

# 直接修改这两个参数即可
PROMPT_TEMPLATE = "本地模型应用场景介绍" # 自定义提示词
COMPLETION_TEMPLATE = f"""
<think>
推理过程...
</think>

根据上下文内容，14b 模型适用于以下本地应用场景：
""".strip() # 自定义完成词

def generate_jsonl():
    """生成包含训练数据的JSONL文件"""
    data_entry = {
        "prompt": PROMPT_TEMPLATE,
        "completion": COMPLETION_TEMPLATE
    }

    with open('prompt_output.jsonl', 'a', encoding='utf-8') as jsonl_file:
        json.dump(data_entry, jsonl_file, ensure_ascii=False)
        jsonl_file.write('\n')

if __name__ == "__main__":
    generate_jsonl()

```

- 执行脚本后，会将内容追加至 prompt_output.jsonl 后(训练要求最少准备四条以上，所以可以重复执行)
- 完成后，将内容 copy 至 valid.jsonl 中即可

3.5 开始微调

1. 执行微调脚本

```
mlx_lm.lora --model ../../DeepSeek-R1-Distill-Qwen-1.5B --train --data ./data
```

以下为一次成功的输出，默认执行 1000 次，每 100 次保存一次 ./adapters 中

```

Loading pretrained model
Loading datasets
Training
Trainable parameters: 0.061% (1.090M/1777.088M)
Starting training..., iters: 1000
Iter 1: Val loss 3.775, Val took 7.053s
Iter 10: Train loss 3.719, Learning Rate 1.000e-05, It/sec 0.457, Tokens/sec 370.97
Iter 20: Train loss 3.448, Learning Rate 1.000e-05, It/sec 0.543, Tokens/sec 378.66
Iter 30: Train loss 3.222, Learning Rate 1.000e-05, It/sec 0.428, Tokens/sec 337.26
Iter 40: Train loss 3.088, Learning Rate 1.000e-05, It/sec 0.592, Tokens/sec 451.74
Iter 50: Train loss 2.980, Learning Rate 1.000e-05, It/sec 0.468, Tokens/sec 354.74
Iter 60: Train loss 2.844, Learning Rate 1.000e-05, It/sec 0.420, Tokens/sec 345.00
Iter 70: Train loss 2.835, Learning Rate 1.000e-05, It/sec 0.480, Tokens/sec 356.30
Iter 80: Train loss 2.772, Learning Rate 1.000e-05, It/sec 0.462, Tokens/sec 353.47
Iter 90: Train loss 2.657, Learning Rate 1.000e-05, It/sec 0.469, Tokens/sec 362.76
Iter 100: Train loss 2.595, Learning Rate 1.000e-05, It/sec 0.437, Tokens/sec 325.6
Iter 100: Saved adapter weights to adapters/adapters.safetensors and adapters/00001
# ... 省略110 -> 900
Iter 910: Train loss 0.551, Learning Rate 1.000e-05, It/sec 0.383, Tokens/sec 284.5
Iter 920: Train loss 0.566, Learning Rate 1.000e-05, It/sec 0.443, Tokens/sec 343.5
Iter 930: Train loss 0.603, Learning Rate 1.000e-05, It/sec 0.336, Tokens/sec 261.7
Iter 940: Train loss 0.506, Learning Rate 1.000e-05, It/sec 0.393, Tokens/sec 295.3
Iter 950: Train loss 0.566, Learning Rate 1.000e-05, It/sec 0.394, Tokens/sec 300.5
Iter 960: Train loss 0.573, Learning Rate 1.000e-05, It/sec 0.330, Tokens/sec 251.3
Iter 970: Train loss 0.609, Learning Rate 1.000e-05, It/sec 0.298, Tokens/sec 239.6
Iter 980: Train loss 0.575, Learning Rate 1.000e-05, It/sec 0.335, Tokens/sec 251.7
Iter 990: Train loss 0.571, Learning Rate 1.000e-05, It/sec 0.460, Tokens/sec 352.5
Iter 1000: Val loss 0.598, Val took 9.913s
Iter 1000: Train loss 0.591, Learning Rate 1.000e-05, It/sec 5.144, Tokens/sec 4017
Iter 1000: Saved adapter weights to adapters/adapters.safetensors and adapters/0001
Saved final weights to adapters/adapters.safetensors.

```

2. 合并本次训练结果模型

```

mlx_lm.fuse --model ../../DeepSeek-R1-Distill-Qwen-1.5B --adapter-path adapters --s
# 输出
Loading pretrained model

```

生成 lora/DeepSeekR1-1.5B-v1 模型

3. 测试微调后模型

```

mlx_lm.generate --model DeepSeekR1-1.5B-v1 --prompt "如何在mac上安装ollama"

```

四、训练结果接入 Ollama

Ollama 加载本地模型指引

4.1 直接加载

1. 新建一份 ollama Modelfile 配置文件，本次案例在根目录新建 ollama_install.mf 文件，示例内容：

```

# Modelfile generated by "ollama show"
# To build a new Modelfile based on this, replace FROM with:
# FROM deepseek-r1:1.5b

# 关键参数, 指向训练后的模型地址
FROM ./mlx-examples/lora/DeepSeekR1-1.5B-v2

# 官方模板
TEMPLATE """{{- if .System }}{{ .System }}{{ end }}
{{- range $i, $_ := .Messages }}
{{- $last := eq (len (slice $.Messages $i)) 1}}
{{- if eq .Role "user" }}<|User|>{{ .Content }}
{{- else if eq .Role "assistant" }}<|Assistant|>{{ .Content }}{{- if not $last }}<
{{- end }}
{{- if and $last (ne .Role "assistant") }}<|Assistant|>{{- end }}
{{- end }}"""
PARAMETER stop <|begin_of_sentence|>
PARAMETER stop <|end_of_sentence|>
PARAMETER stop <|User|>
PARAMETER stop <|Assistant|>
LICENSE """MIT License

```

Copyright (c) 2023 DeepSeek

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

"""

添加 TEMPLATE，不然会出现消息乱回的情况，可以使
用 `ollama show --modelfile deepseek-r1:1.5b` 获取官方模板

2. 执行 ollama 命令

```
ollama create DeepSeekR1-1.5B-v2 -f ollama_install.mf
# 成功案例输出
...
using existing layer sha256:f4d24e9138dd4603380add165d2b0d970bef471fac194b436ebd50e
writing manifest
success
```

3. 执行 ollama list 即可查看模型与对话

4.2 转换 GGUF

训练内容可以转换成 GGUF 格式

1. 环境准备

使用 llama.cpp

```
git clone https://github.com/ggerganov/llama.cpp.git
cd llama.cpp
pip install -r requirements.txt
```

2. 执行转换，生成 models_gguf/DeepSeekR1-1.5B-v2-f16.gguf

```
python convert_hf_to_gguf.py ../mlx-examples/lora/DeepSeekR1-1.5B-v2 --outfile ../m

# 输出如下表示完成
Writing: 100%|██████████| 3.55G/3.55G [00:13<00:00, 262Mbyte/s]
INFO:hf-to-gguf:Model successfully exported to ../models_gguf/DeepSeekR1-1.5B-v2-f16
```

3. Ollama 加载，参考 3.1，将 FORM 参数指向新生成文件即可.

4. llama.cpp 编译

```
# 安装cmake
brew install cmake
# 编译
cmake -B build
```

5. 本次无量化需求，未做实践，如有需要可试试

<https://github.com/ggerganov/llama.cpp/blob/master/examples/quantize/README.md>

训练过程常用命令

```
# 下载模型
# huggingface-cli download --resume-download [模型名称] --local-dir [本地存储地址]
huggingface-cli download --resume-download deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B --

# 训练模型
# mlx_lm.lora --model [训练基础模型] --train --data [训练数据目录]
mlx_lm.lora --model ../../DeepSeek-R1-Distill-Qwen-1.5B --train --data ./data
# 第二轮训练，以上一次训练结果作为底座
mlx_lm.lora --model DeepSeekR1-1.5B-zhibojian --train --data ./data

# 合并模型
# mlx_lm.fuse --model [基础模型] --adapter-path adapters --save-path [合并后的模型名称]
mlx_lm.fuse --model ../../DeepSeek-R1-Distill-Qwen-1.5B --adapter-path adapters --save-
# 二次训练
mlx_lm.fuse --model DeepSeekR1-1.5B-zhibojian --adapter-path adapters --save-path DeepS

# 对话模型
# 建议在ollama导入后进行测试，能获得更准确的响应
# mlx_lm.generate --model [合并后的模型名称] --prompt "如何在mac上安装ollama"
mlx_lm.generate --model DeepSeekR1-1.5B-v1 --prompt "你是谁"
```

五、效果验证与总结

1. 日常问答测试（两轮训练后）：

```

llm_learn ollama run DeepSeekR1-1.5B-zhibojian:latest
>>> 你喜欢吃什么水果?
我喜欢吃苹果和香蕉。
>>> 你平时喜欢做什么?
我喜欢吃苹果和香蕉。
我平时喜欢吃苹果和香蕉，这两果特别美味。
>>> /clear
Cleared session context
>>> 你平时喜欢做什么?
我喜欢学习和帮助用户。

```

2. 文档知识测试：

- 经过三轮简单测试，发现基于文档内容的问答效果未达预期，建议：
 - 优化训练数据预处理
 - 增加训练轮次(epochs)
 - 调整学习率等超参数

3. 实施建议：

- 微调过程需要多次迭代验证
- 建议使用更大规模的专业领域数据集
- 可尝试不同的量化策略提升推理效率